

Final Report Final Project: Time-series data and application to stock markets

A. Reflection for Task 1

TASK 1.1

In **Task 1.1**, I upgraded the model's 'brain' from looking at just one piece of information to looking at six. Originally, the model likely only used the **Close** price to predict the next day. In Task 1.1, I expanded that to include:

1. **Low**: The lowest price of the day.
2. **High**: The highest price of the day.
3. **Open**: The starting price.
4. **Close**: The final price.
5. **Adjusted Close**: The price adjusted for dividends/splits.
6. **Volume**: How many shares were traded.

What changed technically:

- **Data Shape**: Instead of an input shape of (60, 1) (60 days, 1 feature), the LSTM now receives (60, 6) (60 days, 6 features).
- **Normalization**: I applied the MinMaxScaler across all 6 columns simultaneously so that 'Volume' (which is in millions) doesn't overwhelm 'Price' (which is in hundreds).
- **Better Context**: By giving the model all these features, it can see things like 'high trading volume on a low price day,' which provides much more context than price alone.

Transitioning to a multivariate model (Task 1.1) is a significant step. Here are the most important lessons I took away from it:

1. **Dimensionality & Input Shape**: I learned that an LSTM isn't limited to a 1D line of numbers. The input shape (batch, timesteps, features) is a core concept. By changing features from 1 to 6, I learned how to feed the model a 'snapshot' of multiple variables at each point in time.
2. **The Importance of Feature Scaling**: In Task 1.1, I used MinMaxScaler on all features. I learnt that models are sensitive to magnitude. If I didn't scale, the **Volume** (often in the millions) would have completely ignored the **Price** (often in the hundreds) because the math (gradient descent) would prioritize the larger numbers.
3. **Feature Interaction**: I learn that deep learning models excel at finding non-linear relationships *between* variables. By adding 'High' and 'Low', the model can implicitly learn about **Volatility** (the difference between them), which it couldn't do if it only saw the 'Close' price.
4. **Information vs. Noise**: A key takeaway is that more data isn't *always* better, but in time-series, providing a holistic view of the market (Price + Volume) usually helps the model distinguish between a genuine trend and a random spike.

TASK 1.2

In **Task 1.1**, we were predicting 'tomorrow's' price. In **Task 1.2**, we changed the goal to predict the price **k days from now** (for example, 3 days ahead).

Think of it like this:

- **Standard Model**: Sees data from Monday–Friday and predicts **Saturday**.
- **Task 1.2 Model**: Sees data from Monday–Friday and predicts **Tuesday of next week** ($k=3$).

What I learn from this:

1. **Target Slicing**: I learned how to 'offset' your target variable. Instead of y being the very next row after my sequence, we tell the code to look k rows ahead.
2. **Forecasting Horizon**: This teaches me that the 'difficulty' of a prediction changes. Predicting tomorrow is usually easier than predicting a week from now. If looking at the results, I often see the error (MSE) go up as k increases because the future becomes more uncertain.
3. **Real-world Utility**: In trading, I often need time to execute a strategy. Predicting a price several days out is often more useful than just predicting the next 24 hours.

TASK 1.3

What I learn from Task 1.3:

1. **Multi-Output Regression**: I learned that a model's final layer isn't restricted to one neuron. By setting `Dense(k_consecutive)`, I taught the model to map one history (60 days) to multiple future outcomes. This is a move from 'Many-to-One' to 'Many-to-Many' logic.
2. **Capturing Trends vs. Points**: Unlike Task 1.2 (which only cares about a specific day), Task 1.3 forces the model to learn the **trajectory** of the stock. The loss function (MSE) now penalizes the model if it gets the 'path' of the next 3 days wrong, not just the final destination.
3. **Data Structure for Sequences**: I learned how to restructure my target y into a 2D array (a vector for each sample). This is essential for any real-world forecasting tool where a user wants to see a 'forecast line' rather than just a single future price.
4. **Efficiency**: Predicting 3 days in one single 'forward pass' of the model is much faster and more consistent than running a 1-day model three times and feeding the results back into itself (which often leads to error accumulation).

(And from what the professors suggested) **The core of time-series best practices:**

1. Training / Validation / Test Split (Time-Conforming)

- **The Lesson: Never use future data to predict the past.**
- In standard ML, I can shuffle data randomly. In time-series, I must maintain chronological order. If I shuffle, the model might see 'future' prices during training, leading to an artificially high accuracy that fails completely in real-world trading.

2. Cross-Validation (Time-Conforming)

- **The Lesson: Traditional K-Fold CV is broken for time-series.**
- Standard K-Fold picks random chunks for validation. Instead, I must use **Expanding Window** or **Sliding Window** CV (like `TimeSeriesSplit`). This ensures that for every 'fold', the training set always occurs before the validation set in time.

3. Time Windows (1-Month Train / 1-Week Test)

- **The Lesson: Models 'decay' as markets change.**
- By using a sliding window (training on the last month to predict the next week), I learn that financial patterns are **non-stationary**—meaning what worked in 2020 likely won't work in 2024. This teaches me how to implement 'walk-forward' validation, which mimics how a real trading bot would re-train itself periodically.

4. Company Filtering

- **The Lesson: Quality over Quantity (Data Robustness).**
 - **The 120-point rule:** Deep learning models (like LSTM) need a minimum amount of history to detect patterns. Including 'new' companies with only 10 days of data introduces noise.
 - **Sector/Exchange Filtering:** Companies in the same industry (e.g., Tech vs. Energy) react differently to the same news. Learning to filter allows me to build **Specialized Models** rather than a 'one-size-fits-all' model that is mediocre at everything.

In short: These points teach me how to avoid 'Data Leakage' and how to build a model that is robust enough to survive the unpredictability of the stock market.

B. Reflection for Task 2

TASK 2.1

1. Feature Interaction: Stock prices don't move in a vacuum. Learning that **Volume** (liquidity) and the spread between **High/Low** are often leading indicators for price movements is crucial. By using multiple features, the model learns the *relationship* between these variables.

1. **Multivariate Input Handling:** I learned how to restructure data into a 3D shape: (Samples, Time_Steps, Features). This is the standard for Recurrent Neural Networks (RNNs/LSTMs).
2. **Data Alignment:** I learned how to ensure that the features at time t are used to predict the target at $t+1$, ensuring that I am not accidentally 'leaking' future information into the features.
3. **Normalization Scales:** Since Volume is usually in the millions and Price is in the thousands, I learned (via MinMaxScaler) that neural networks fail if features are not scaled to a similar range (usually 0 to 1).

TASK 2.2

Task 2.2 moves from predicting 'tomorrow' to predicting a specific day in the future (e.g., $k = 7$). This shifts the model from a simple autoregressive task to a **horizon forecast**.

1. Challenges in implementing Task 2.2

- **Increasing Variance:** As the prediction horizon k increases, the statistical relationship between the input window and the target weakens. Predicting 7 days ahead is significantly harder than 1 day because of market noise.
- **Window Management:** I had to carefully adjust your data slicing logic to ensure the target y was exactly k days ahead of the end of the input window X without overlapping or skipping samples incorrectly.

- **Data Sufficiency:** If a company has a short history, shifting the target by k days reduces the total number of training samples available.

2. Overall Workflow of Task 2.2

In this task, the code:

- Sets a defined look-ahead parameter ($k=7$).
- Prepares a dataset where each input block of 60 days is paired with a single price point 7 days in the future.
- Trains a specific LSTM model instance (`model_k_ahead`) dedicated to this horizon.
- Visualizes the results by shifting the prediction line relative to the real price to see if the model successfully 'anticipated' the move 7 days early.

3. Technical Changes (What changed?)

- **Target Slicing:** In `prepare_input`, the index for y changed from $i + \text{window_size}$ to $i + \text{window_size} + k - 1$.
- **Look-back Increase:** I increased the `window_size` from 30 to **60**. This is a technical necessity: to predict further into the future, the model usually needs a longer historical context to identify broader trends.
- **Temporal Mapping:** Technically, the loss function now measures the error over a 'gap'. The model is forced to ignore immediate fluctuations and focus on 'lagged' correlations.

4. Learned Lessons

- **Predictability Decay:** I likely noticed the MSE (Mean Squared Error) is higher for Task 2.2 than Task 2.1. This teaches me that information 'evaporates' over time in financial markets.
- **The Lag Effect:** Often, models predicting k -days ahead end up simply 'mimicking' the last known price. Learning how to check if my model is actually predicting a trend or just repeating the last value is a vital lesson in time-series.
- **Hyperparameter Sensitivity:** I learned that the same architecture used for 1-day prediction might need more depth or a different learning rate when the objective becomes more difficult (7-day prediction).

TASK 2.3

Task 2.3 is the most complex of the forecasting tasks because it transitions from predicting a single point to predicting a **sequence** of 15 consecutive days.

1. Challenges in implementing Task 2.3

- **Error Accumulation:** In multi-step forecasting, errors in predicting the 1st day can propagate and magnify by the 15th day.
- **Loss Function Design:** A standard MSE treats the 1st day and 15th day with equal importance. However, in trading, the 1st day is much more actionable. Creating a loss function that balances this was a significant hurdle.
- **Dimensionality and Complexity:** The model output shape changes from (1,) to (15,), requiring the Dense layer to map high-level LSTM features to a multi-dimensional target vector.

2. Overall Workflow of Task 2.3

In this task, the code:

- Sets $k = 15$ and uses a 60-day input window.
- Modifies the data preparation to create targets (y) that are vectors of 15 prices.
- Implements a specialized training loop using a **Custom Loss Function**.
- Visualizes the 'Forecast Path' to show how the model expects the stock to behave over a two-week period.

3. Technical Changes (What changed?)

- **The Custom Loss Function:** I implemented `exponential_decay_weighted_mse`. This is a sophisticated technical change that uses `tf.pow(decay_rate, tf.range(k))` to weight the first few days of the prediction more heavily than the later days.
- **Output Layer:** The final Dense layer was modified to have `units = k (15)`, allowing the network to output 15 separate values simultaneously.
- **Data Preparation:** The `consecutive=True` flag in your function changed the target y from a scalar value to a slice: `data["Open"].iloc[i+window_size : i+window_size+k]`.

4. Learned Lessons

- **Recency Bias in Forecasting:** I learned that for long-term predictions to be useful, the model must prioritize the near-term path.
- **Multi-Output Regression:** I learned how neural networks can perform 'Many-to-Many' or 'Many-to-Vector' mapping, which is a step up from simple regression.
- **Vectorized Tensor Operations:** By writing the custom loss using `tf.reduce_mean` and `weights[None, :]`, I learned how to perform efficient batch operations in TensorFlow without slow Python loops.

General lessons:

1. Time Window (Look-back)

- **The Lesson:** The window size (30 vs 60 days) is a trade-off between signal and noise.
- **Reflection:** I learned that for a 1-day prediction, a short 30-day window might suffice to capture local momentum. However, for 7-day or 15-day forecasts, I increased the window to 60 days. This teaches me that the 'memory' of the model must be proportional to the 'horizon' of the forecast.

2. Company Filtering

- **The Lesson:** Not all stocks are predictable. Low-liquidity stocks or those with short histories produce 'trash' data.
- **Reflection:** By filtering for banks with stock ratings ≥ 3.5 and at least 10 years of existence, I ensured the model was trained on 'High-Quality Signal' data. I learned that the quality of my dataset is often more important than the complexity of my LSTM architecture.

3. Additional Vietnam-Specific Data

- **The Lesson:** Price alone doesn't tell the whole story.
- **Reflection:** I learned that using **Financial Ratios** (P/B, ROA) and **Dividend History** allows me to filter for 'Fundamental Stability.' In the volatile Vietnam market, a company that pays

consistent dividends is often more 'predictable' and less prone to speculative manipulation, providing a safer environment for your LSTM to learn patterns.

C. Reflection for Task 3

TASK 3.1

The Justification for the Model Design

1. **Hybrid Approach (Rules + Pattern Recognition):** The model is built on the philosophy that while simple technical indicators (SMA, RSI, MACD) are good at identifying general market conditions, they often produce 'false positives' in isolation. By training an **LSTM** on these signals, I am teaching the model to recognize the specific *shape* of the price movement that historically led to a successful signal.
2. **Selection of Binary Cross-Entropy:** Since I am identifying a discrete event (an Entry Point), I treated the problem as a **probabilistic classification**. This allows the model to output a 'Confidence Score' rather than a hard yes/no.
3. **Use of Temporal Memory:** Use **LSTM** architecture because entry points are rarely about a single day's price; they are about the **sequence** of lower lows or decreasing volume leading up to that point. The LSTM's hidden state captures this 'context' better than a simple logistic regression.
4. **MACD Filtering for Signal Reliability:** I implemented a filter: $\text{abs}(\text{macd_value}) > \text{macd_threshold}$. I only want the model to learn from 'Significant Market Events.' Training only on days with strong momentum (high MACD) prevents the model from being confused by the 'sideways' or flat market noise typical of less liquid periods.

Task 3.1 transitions from a **Regression** problem (predicting a price) to a **Classification** problem (predicting an action).

1. Challenges in implementing Task 3.1

- **Class Imbalance:** In a typical stock history, 'Buy' signals are rare compared to 'Hold' or 'Neutral' days. This makes it hard for the model to learn because it sees many more 0s than 1s.
- **Label Subjectivity:** Unlike a future price (which is an objective fact), a 'Buy Signal' is based on rules I defined. If my rules (SMA/RSI/MACD) are too strict, I have no data; if they are too loose, the model learns to buy during 'noise'.
- **Threshold Selection:** Deciding the probability threshold (e.g., > 0.5) to actually trigger a 'Buy' alert is difficult and significantly affects the Precision vs. Recall balance.

2. Overall Workflow of Task 3.1

In this task, the code:

- **Engineers Features:** Uses the `ta` library to calculate technical momentum and trend indicators.
- **Labels Data:** Applies a conditional logic function to scan the historical data and mark days that meet the 'Buy' criteria (e.g., price at 20th percentile + $\text{RSI} < 50$).
- **Filters Samples:** Uses a MACD-based filter ($\text{abs}(\text{macd_value}) > \text{macd_threshold}$) to only train the model on days where there is significant market movement.
- **Trains Classifier:** Fits an LSTM with a sigmoid output to predict the probability of a signal.

3. Technical Changes (What changed?)

- **Activation Function:** The final layer changed from None (linear) to sigmoid. This squashes the output to a range between 0 and 1, representing a probability.
- **Loss Function:** I switched from MSE (Mean Squared Error) to `binary_crossentropy`. This is mathematically necessary for classification as it penalizes wrong 'yes/no' guesses much more effectively.
- **Metrics:** I started monitoring accuracy and precision instead of just error distance, as I now care about how many 'Buy' alerts were actually correct.

4. Learned Lessons

- **Filtering for Quality:** I learned that training on every day is inefficient. By filtering with MACD, I learned that focusing the model on 'volatile' or 'interesting' segments of data improves the signal-to-noise ratio.
- **Rules vs. Intelligence:** I learned that the model is only as good as my labeling logic. If the RSI/SMA rules are bad, the model will faithfully learn a bad strategy.
- **Probability vs. Binary:** I learned that models don't just say 'Buy'; they give a confidence score. Learning to interpret a 0.6 vs a 0.9 probability is a key skill in algorithmic trading.

TASK 3.2

Task 3.2 (Signal Identification Model) development:

1. Problem Framing: Supervised Sequence Classification

Instead of simple linear triggers, I built this as a deep learning classification problem to account for the **temporal context** of market movements. While a standard indicator is reactive, my model learns the 'pre-conditions' (the sequence of candles) that traditionally lead to successful trades.

2. Labeling Strategy (Expert Rule-Based Ground Truth)

- **Percentile-Based Bounds:** I used 20/80 price percentiles within a 30-day window to identify locally 'cheap' and 'expensive' zones. This justifies the model's objective as finding **Mean Reversion** opportunities.
- **Indicator Confluence:** I required signals to align with **RSI** (momentum exhaustion) and **MACD** (trend shift). This 'double-confirmation' logic reduces noise in the training data, ensuring the model only learns from high-conviction entries.

3. Architecture Choice: LSTM (Long Short-Term Memory)

- **Memory of Trends:** I chose LSTMs because price signals are path-dependent. An RSI of 30 after a sharp crash means something different than an RSI of 30 after a slow grind. The LSTM cells capture this historical 'shape' through their hidden states.
- **Sigmoid Activation:** Since the goal is a Buy or Sell probability, the **Sigmoid** output layer is used to produce a confidence score (0 to 1), allowing you to set a 'threshold of certainty' (e.g., only trading when probability > 0.7).

4. Momentum Filtering for Quality Control

- **MACD Absolute Thresholding:** In my code, I filtered data using `abs(macd_value) > macd_threshold`. I can justify this by stating that it prevents the model from being 'poisoned' by sideways/flat market data, focusing its learning power solely on **active volatility events**.

5. Handling Class Imbalance

- By using **Binary Cross-Entropy**, I penalized the model for missing these rare 'Buy' events more effectively than a standard accuracy metric would, which is essential given that 'Buy' signals occur much less frequently than 'Hold' periods.

Other aspects:

1. Implementation Challenges

- **Class Imbalance:** In financial markets, 'Buy' and 'Sell' opportunities are rare. Most days are 'Hold' days. This imbalance makes it difficult for the model to learn signals without simply guessing 'Hold' every time.
- **Signal Lag:** Technical indicators like SMA and MACD are lagging by nature. Reconciling these lags with a predictive LSTM requires careful alignment of the look-back windows.
- **Expert Rule Complexity:** Designing a function (`generate_signals`) that balances sensitivity (finding signals) and specificity (avoiding bad trades) requires multiple indicator confluences (RSI + MACD + SMA).

2. Overall Code Logic

- **Manual Feature Engineering:** The code uses the `ta` library to calculate SMA, RSI, and MACD.
- **Labeling (The 'Teacher'):** It creates a ground truth by looking at future prices within a 20-day window. If the current price is in the bottom 20% and has a high potential for profit, it labels it as a 'Buy'.
- **Training Classification Models:** It trains two separate LSTMs (`model_buy` and `model_sell`) to recognize the 20-day price 'signature' that typically precedes these expert labels.

3. Technical Changes (What was new?)

- **Classification vs. Regression:** Task 2 predicted a continuous number (price). Task 3.2 switched to **Binary Classification**, using **Sigmoid activation** and **Binary Cross-Entropy**.
- **Input Dimensionality:** The model shifted from multi-feature inputs (OHLCV) to a sequence of normalized prices, focusing specifically on the relative shape of the trend.
- **Threshold Filtering:** I introduced a dynamic MACD threshold (`MACD_abs_mean`) to filter out training data from low-volatility periods, technically acting as a 'Denoising' step.

4. Lessons Learned

- **Hybrid Models are Stronger:** I learned that LSTMs perform better when the 'search space' is narrowed down by fundamental/technical rules rather than trying to learn from raw, unfiltered noise.

- **Probability is Better than Binary:** By using Sigmoid, I learned that the model provides a *degree of certainty*. This is crucial for risk management—one should only trade when the model is >80% sure.
- **The Importance of Evaluation:** I observed that while 'Accuracy' might look high (because 'Hold' is common), 'Precision' and 'Recall' are the true measures of success for trading signals.

Reflections

1. Time Window for Training and Testing

- **Lesson:** There is a 'Goldilocks' zone for window sizes. In my code, I experimented with windows like 20, 30, and 60 days. I learned that too small a window lacks enough context to identify a trend, while too large a window might introduce obsolete patterns that no longer apply to current market volatility.

2. Sophisticated Company Filtering

- **Lesson:** Deep learning requires **quality over quantity**. By filtering for 10+ years of history, financial ratios (ROE, ROA), and dividend stability, I essentially performed 'Denoising.' I learned that a model trained on healthy, liquid companies (like the 16 Banks) is much more reliable than a model trying to find patterns in 'penny stocks' or highly manipulated low-cap companies.

3. Manual Feature Engineering (SMA, MACD, RSI)

- **Lesson:** It is highly beneficial. While LSTMs can theoretically learn these patterns from raw price, using indicators as 'Expert Rules' for labeling provides the model with a **clear objective**. Instead of asking the model to find 'meaning' in noise, I used engineering to say: 'Learn the price signature that leads to this specific technical setup.' This hybrid approach significantly improves the Signal-to-Noise ratio.

D. REFLECTION FOR TASK 4

TASK 4.1

1. Methodology: Profit Estimation & Optimization

- **Profit Estimation:** I utilized the LSTM model from Task 2 to predict the stock price 7 days into the future. By comparing this predicted price against the current market price, the code calculates the **Expected Return** (the 'Alpha').
- **Portfolio Optimization:** I moved beyond simple selection by creating a **Combined Index**. This methodology weights the model's reliability (1 - MSE) against historical performance (Sharpe Ratio), ensuring that a stock is selected not just because it might go up, but because the model is confident and the history is stable.

2. Implementation Challenges

- **Scaling Inconsistency:** Each company has a different price scale (some trade at 10k, others at 100k). Re-normalizing these individual predictions using the MinMaxScaler for each specific ticker to get a real currency value was a significant data-handling hurdle.

- **Processing Overhead:** Evaluating the LSTM models across all 16 banks individually to generate a summary table is computationally expensive compared to simple statistical models.

3. Overall Code Logic

- **The Pipeline:** The code iterates through the list of banks, loads their historical CSVs, feeds the last 60 days of data into the LSTM, denormalizes the output to get a price, and then ranks the companies in a `results_df` based on the efficiency of the return relative to the risk.

4. Technical Changes

- **Model Deployment:** In Task 2, I was *building* the models. In Task 4.1, you shifted to *deploying* them to make real business decisions.
- **Inclusion of the Sharpe Ratio:** You introduced the **Sharpe Ratio** (Mean Return / Volatility), adding a classical financial metric to your deep learning results to manage portfolio risk.

5. Lessons Learned

- **MSE is as important as Profit:** I learned that a high profit prediction is useless if the model's error (MSE) is also high. The 'best' stocks are often those where the model is the most accurate.
- **Normalization is Context-Specific:** I learned that when moving from training to inference, the scalers (`sc`) must be carefully tracked for each individual company to avoid 'value explosion' or incorrect profit calculations.
- **Diversification is Automated:** By using the optimization logic, I realized that the best portfolio isn't necessarily just the #1 stock, but a mix that balances the strengths and weaknesses of multiple tickers.

TASK 4.2

For **Task 4.2 (Risk Management)**, my methodology used a dual-layered risk scoring system to identify and exclude unstable companies:

1. Historical Risk (The Sharpe Ratio)

- The primary risk score is derived from the **volatility of historical returns**. By calculating the standard deviation of daily percentage changes, the code identifies stocks with high 'market noise'.
- A low or negative Sharpe Ratio serves as a high-risk flag, indicating that the potential return does not justify the historical price swings (the risk).

2. Prediction Risk (Mean Squared Error)

- I introduced **Model Uncertainty** as a risk metric. If the LSTM model has a high MSE for a specific company, it means the patterns of that stock are erratic and unpredictable.
- This justifies exclusion because even if a stock *looks* profitable, the risk of the 'prediction being wrong' is mathematically too high to include it in a managed portfolio.

3. The Combined Risk Score (The Index)

- In my `filter_companies` logic, I combined these into a weighted score: $(1 - \text{MSE}) * 0.4 + \text{Sharpe} * 0.6$.

- **Risk Exclusion Criteria:** Any company that ends up in the bottom of this ranking is effectively 'voted out' by the risk model. For instance, **SHB** and **ACB** were excluded because their combined score was significantly lower than the market average, indicating either poor historical stability, high model error, or both.

This methodology ensures that 'Risk' is defined not just by price drops, but by the **unreliability** of the asset.

Other aspects:

1. Challenges in Implementing Task 4.2:

- **Multi-Factor Balancing:** The primary challenge was creating a meaningful ranking system that didn't just look at returns. It had to mathematically balance **Model Reliability** (Inverse MSE) with **Historical Performance** (Sharpe Ratio).
- **Data Alignment:** Ensuring that the model predictions for 16 different banks, each with varying historical windows, were normalized and comparable for the ranking index.

2. What the code did overall in Task 4.2:

- It implemented a `filter_companies` function that iterates through the list of banks and performs a multi-step evaluation for each:
 - Calculates the **Sharpe Ratio** (historical risk-adjusted return).
 - Evaluates the specific **MSE (Mean Squared Error)** for that ticker using the trained model.
 - Generates a **7-day price prediction** to calculate an **Expected Return**.
 - Computes a **Combined Index** as a weighted average: $(1 - \text{MSE}) * 0.4 + \text{Sharpe_Ratio} * 0.6$.

3. What changed technically in Task 4.2:

- **Hybrid Ranking:** The code moved beyond standard financial filtering (Task 1) to a 'Hybrid Intelligence' approach where the decision to include a stock depends on the **AI's confidence** (MSE) in that specific ticker.
- **Normalization:** It used the `split` and `denorm` functions within the filtering loop to ensure the MSE and expected returns were calculated on the same scale for all 16 banks.

4. Learned Lessons:

- **Model-Specific Risk:** A key takeaway is that the model is better at predicting some stocks than others. Using a 'Combined Index' prevents the portfolio from selecting high-growth stocks that the model cannot accurately predict.
- **Expected Return Divergence:** The code revealed that even within the same sector (Banks), expected returns can vary wildly (~41% for KLB vs -64% for ACB), highlighting the need for active filtering before optimization.

Task 4.3: Optimal Investment Allocation

1. Challenges in Implementing Task 4.3:

- **Matrix Dimensionality:** A significant challenge was ensuring the covariance matrix and expected returns vector were compatible, especially when dealing with the limited subset of ranked stocks.
- **Mathematical Convergence:** Designing the optimization to minimize the *negative* Sharpe Ratio (since most optimizers are minimizers) required careful handling of the objective function to avoid errors when volatility approached zero.
- **Real-world Constraints:** Implementing 'No Short Selling' (weights between 0 and 1) and 'Full Investment' (sum of weights = 1) required defining specific linear constraints for the SLSQP algorithm.

2. What the code did overall in Task 4.3:

- It performed **Mean-Variance Optimization (Markowitz)**.
- The code defines a `portfolio_optimization` function that calculates the weights for each asset that result in the highest possible risk-adjusted return (Sharpe Ratio).
- It used the **Expected Returns** generated by your LSTM model in the previous step and a **Covariance Matrix** of those returns to determine asset correlations.

3. What changed technically in Task 4.3:

- **Shift from Heuristic to Mathematical:** Technically, the code moved from simple ranking (Task 4.2) to **quadratic programming**.
- **Dynamic Covariance:** Instead of assuming assets move independently, the code calculates `np.cov` on the returns, allowing the optimizer to reduce portfolio risk by picking assets that are not perfectly correlated.
- **Numerical Optimization:** It introduced the `scipy.optimize.minimize` library using the SLSQP method, which is a significant technical step up from basic data manipulation.

4. Learned Lessons:

- **Concentration Risk:** The optimizer often defaults to a 'corner solution' (allocating 100% to one asset, like OCB in your output) if that asset's expected return significantly outweighs the others. This teaches the importance of adding 'Maximum Weight' constraints (e.g., no more than 30% per bank) to ensure true diversification.
- **Garbage In, Garbage Out:** The optimization results are extremely sensitive to the LSTM's predictions. If the model is overly optimistic about one ticker, the optimizer will aggressively pivot toward it, highlighting that portfolio quality is only as good as the underlying forecasting model.

